Offline Model-Based Planning on Experience Transition Graphs for RTS Micromanagement

Anonymous

Anonymous

Abstract

Real-time strategy (RTS) micromanagement presents a challenging decision-making problem with high-dimensional, continuous state spaces and stochastic outcomes. While deep reinforcement learning has achieved notable success in game AI, its black-box nature limits interpretability and reproducibility—critical requirements for strategic analysis and deployment. We propose **Offline Model-Based Planning on Experience Transition Graphs (ETG)**, a framework that decouples data collection from decision-making by constructing a lightweight, explicit world model directly from RL interaction trajectories. The ETG is a directed weighted graph where nodes represent abstracted game states, edges encode action-labeled transitions with multi-dimensional statistical attributes (visit counts, win rates, quality scores), and these statistics provide empirical approximations of MDP transition and reward functions. We perform **beam search planning** on the ETG with a four-dimensional path evaluation framework (confidence, quality, win rate, reward), and introduce a **Branch-to-Backup (BTB) recovery mechanism**—inspired by CPU branch prediction—that enables efficient plan adaptation when execution diverges from the predicted trajectory. Evaluated on StarCraft II micromanagement scenarios, the ETG-based planner achieves a **96.4%** historical win rate on recommended actions and an **87%** simulation win rate over 100 episodes, while providing fully interpretable and reproducible decision paths.

Introduction

Real-time strategy (RTS) games are widely regarded as a challenging benchmark for artificial intelligence, requiring agents to manage high-dimensional continuous state spaces, long-horizon planning, and real-time decision-making under uncertainty (Ontañón et al. 2013; Vinyals et al. 2019). In the micromanagement sub-task—controlling individual combat units to maximize combat effectiveness—the state space grows exponentially with the number of units, and the optimal action at each step depends on complex spatial configurations of friendly and enemy forces (Vinyals et al. 2019; Samvelyan et al. 2019).

Recent advances in deep reinforcement learning (RL) and offline policy learning have demonstrated strong performance on RTS micromanagement benchmarks (Vinyals et al. 2019; Samvelyan et al. 2019). However, these approaches face inherent limitations: **(1)** neural network policies are black boxes—users cannot trace *why* a particular action was recommended or explore counterfactual alternatives; **(2)** policy learning requires iterative gradient updates with non-deterministic outcomes, making results difficult to reproduce; **(3)** modifying the objective (e.g., trading off aggression for safety) requires retraining.

An alternative paradigm, **Offline Model-Based Planning**, constructs an explicit model of the environment from historical data and performs planning via search rather than policy optimization (Argenson and Dulac-Arnold 2021; Janner et al. 2021). This approach offers deterministic, reproducible, and interpretable decision-making, but existing methods typically rely on learned neural network dynamics models (Argenson and Dulac-Arnold 2021; Zhan et al. 2022) or latent-space graph representations (Zhang, Yang, and Stadie 2021; Zhu et al. 2023), which reintroduce opacity and training complexity.

In this paper, we propose the **Experience Transition Graph (ETG)**—a tabular, explicitly interpretable world model constructed by statistical aggregation of RL trajectories. The ETG stores state-action statistics (visit counts, win rates, quality scores, transition frequencies) that directly approximate MDP transition and reward functions, without any neural network training.

On top of the ETG, we develop three core components:

- **Beam Search Planning**: A constrained forward search that evaluates candidate action sequences using a four-dimensional scoring framework (confidence, quality, win rate, reward) with Min-Max normalization.
- **Branch-to-Backup (BTB) Recovery**: Inspired by CPU branch target buffers, a pre-computed switching map that enables seamless path adaptation during receding-horizon rollout when execution diverges from the plan.
- **Receding-Horizon Rollout**: A Plan-Execute-Diverge loop that combines multi-step plan execution with on-demand re-planning at divergence points.

Our key contributions are:

1. We formalize the **Experience Transition Graph** as a lightweight, explicit world model for RTS micromanagement, with rigorous probabilistic semantics connecting its statistics to MDP components.

Copyright © 2024, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

2. We design a **beam search planner** with a four-dimensional path evaluation framework that balances confidence, quality, win rate, and reward.
3. We introduce the **Branch-to-Backup (BTB) recovery mechanism**, a novel planning adaptation strategy inspired by CPU branch prediction that pre-computes alternative path switching points for efficient divergence handling.
4. We demonstrate the effectiveness of the full pipeline on StarCraft II micromanagement, achieving 96.4% historical win rate on recommended actions and 87% simulation win rate, with fully interpretable decision traces.

Related Work

Graph-Structured World Models

The idea of using graphs as world models has gained increasing attention. Zhang, Yang, and Stadie (2021) propose L3P, which constructs a graph of latent landmarks from Q-functions and performs graph search for long-horizon planning. Zhu et al. (2023) introduce the Value Memory Graph (VMG), which builds a directed graph MDP from offline datasets with graph states and graph actions, performing value iteration directly on the graph. de Mendonça, de Barros Costa, and Prudêncio (2018) propose Abstract State Transition Graphs (ASTG) by fusing structurally similar states for model-based RL. Yin et al. (2024) construct "highway" structures on experience transition graphs to compress models and accelerate value propagation. Our ETG differs from these approaches in its use of multi-dimensional statistical attributes (quality scores, win rates, future rewards) on each edge, enabling richer planning heuristics than transition probabilities alone.

Offline Model-Based Planning

Model-Based Offline Planning (MBOP) (Argenson and Dulac-Arnold 2021) learns a dynamics model from offline data and performs planning via shooting methods, without online interaction or policy learning. MOPP (Zhan et al. 2022) extends MBOP with trajectory pruning guided by behavior policies to avoid out-of-distribution actions. The Trajectory Transformer (Janner et al. 2021) frames offline RL as sequence modeling and uses beam search over trajectory distributions for planning. These methods rely on learned neural network dynamics models or large-scale transformers. In contrast, our ETG is a purely tabular model constructed by statistical aggregation, offering deterministic and interpretable planning without neural network training.

State Abstraction

State abstraction is essential for scaling RL to complex domains (Starre, Saccenti, and Ferreira 2022). Li et al. (2022) establish the sample complexity of model-based offline RL with empirical transition matrices in tabular MDPs, providing theoretical grounding for count-based transition models like the ETG. BK-Trees (Burkhard and Keller 1973) provide an efficient metric-space indexing structure that we adapt for hierarchical state clustering in the RTS domain, combined with Hungarian matching (Kuhn 1955) for unit-to-unit distance computation under varying army sizes.

RTS Game AI

StarCraft II has become a standard benchmark for multi-agent RL (Vinyals et al. 2019; Samvelyan et al. 2019). SMAC (Samvelyan et al. 2019) provides decentralized micromanagement scenarios benchmarked by QMIX and various centralized/decentralized methods. Our work addresses a complementary objective: rather than learning optimal policies, we construct an interpretable planning model from RL experience that supports human-understandable decision traces and counterfactual analysis.

Method

Problem Formulation

We model the RTS micromanagement problem as a Markov Decision Process (MDP) $\mathcal{M} = (\mathcal{S}, \mathcal{A}, T, R, \gamma)$, where $\mathcal{S}$ is the continuous state space (unit positions, health values, cooldowns), $\mathcal{A}$ is the discrete action space (movement and attack commands), $T(s'|s,a)$ is the transition function, $R(s,a)$ is the reward function, and $\gamma \in [0,1)$ is the discount factor.

Given a dataset $\mathcal{D} = \{\tau_1, \ldots, \tau_N\}$ of RL interaction trajectories $\tau_i = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots)$ collected by a trained RL agent, our goal is to construct an explicit planning model and perform **offline planning**: given a current state s_0, search for an optimal action sequence without further environment interaction.

State Abstraction via Hierarchical Clustering

The continuous RTS state space requires discretization for tabular modeling. We employ a two-level BK-Tree (Burkhard and Keller 1973) hierarchy for state abstraction.

State Representation. A game state $s \in \mathcal{S}$ is represented by the configuration of friendly and enemy units: $s = (\mathbf{b}, \mathbf{r})$ where $\mathbf{b} = \{(x_i^b, y_i^b, h_i^b)\}_{i=1}^{n_b}$ is the blue army (friendly units) and $\mathbf{r} = \{(x_j^r, y_j^r, h_j^r)\}_{j=1}^{n_r}$ is the red army (enemy units), with coordinates (x, y) and health points h.

Unit Matching Distance. To compare states with different unit arrangements, we define a matching-based distance using the Hungarian algorithm (Kuhn 1955):

$$d(s, s') = d_{\text{pos}}(s, s') + \alpha \cdot d_{\text{hp}}(s, s') \tag{1}$$

where d_{pos} computes the minimum-cost bipartite matching between unit positions and d_{hp} computes the corresponding health value differences, with α balancing spatial and health proximity.

Hierarchical BK-Tree Clustering. A primary BK-Tree clusters states at a coarse granularity, and secondary BK-Trees provide finer sub-clustering within each primary cluster. Each unique state is assigned a discrete identifier $\hat{s} = (\text{primary_id}, \text{secondary_id}) \in \hat{\mathcal{S}}$.

Figure 1: System Architecture

RL Trajectory Collection → BK-Tree State Abstraction → ETG Construction → Beam Search Planning → BTB Recovery & Rollout → Visualization

Figure 1: **Overview of the ETG-based planning framework.** RL agent trajectories are collected and abstracted into discrete states via BK-Tree clustering. Statistics are aggregated into an Experience Transition Graph. Beam search planning on the ETG produces recommended actions, with BTB recovery handling execution divergence during receding-horizon rollout.

Experience Transition Graph Construction

The Experience Transition Graph (ETG) is a directed weighted graph $G = (V, E, \phi)$ defined as follows:

Nodes $V \subseteq \hat{\mathcal{S}}$: Each node represents an abstracted state identifier assigned by the BK-Tree hierarchy.

Edges $E \subseteq V \times \mathcal{A} \times V$: Each directed edge (s, a, s') represents an observed state-action-next-state transition.

Attribute Function $\phi : E \to \mathbb{R}^k$: Each edge carries a k-dimensional vector of statistical attributes computed from the trajectory data.

For each state-action pair (s, a) observed in the dataset, we compute:

$$\text{visits}(s, a) = N(s, a) = \left|\{\tau_i : \exists t, \tau_i.s_t = s, \tau_i.a_t = a\}\right| \tag{2}$$

$$\hat{T}(s'|s, a) = \frac{N(s, a, s')}{N(s, a)} \tag{3}$$

$$\hat{R}(s, a) = \frac{1}{N(s, a)} \sum_{(s,a)\in\mathcal{D}} r_{s,a} \tag{4}$$

$$\hat{Q}(s, a) = \frac{1}{N(s, a)} \sum_{(s,a)\in\mathcal{D}} G_{s,a} \tag{5}$$

$$\hat{w}(s, a) = \frac{|\{\text{episodes with win} \mid (s, a)\}|}{N(s, a)} \tag{6}$$

where $\hat{T}$ is the empirical transition probability, $\hat{R}$ is the average immediate reward (step reward), $\hat{Q}$ is the average cumulative future reward, and $\hat{w}$ is the empirical win rate.

The **quality score** combines future reward and win rate:

$$q(s, a) = 0.7 \cdot \hat{Q}(s, a) + 30.0 \cdot \hat{w}(s, a) \tag{7}$$

Table 1 summarizes the probabilistic semantics of ETG attributes. The ETG thus serves as a *tabular empirical MDP*—the simplest form of an explicit world model that requires no neural network training (Li et al. 2022).

Beam Search Planning on ETG

Given a starting state s_0 and the ETG, we perform multi-step constrained forward search via beam search to find the optimal action sequence.

Table 1: **ETG statistics and their MDP semantics.** Each statistical attribute approximates a component of the underlying MDP.

ETG Attribute	Statistic	MDP Approximation
visits	$N(s, a)$	$\pi(a\|s) \propto N(s, a)$
transitions	$N(s, a, s')$	$T(s'\|s, a)$
avg_step_reward	$\bar{r}(s, a)$	$R(s, a)$
avg_future_reward	$\bar{G}(s, a)$	$Q^*(s, a)$
win_rate	$\hat{w}(s, a)$	Policy quality
quality_score	$q(s, a)$	Utility $U(s, a)$

Algorithm 1 describes the beam search procedure. At each depth d, all active beams are expanded by considering candidate actions ranked by the ETG quality statistics. The **scoring function** f supports three modes: *quality* (Equation 6), *future reward* ($\hat{Q}$), and *win rate* ($\hat{w}$).

Four-Dimensional Path Evaluation. Each candidate path $p = [(s_0, a_0), (s_1, a_1), \ldots, (s_L, a_L)]$ is evaluated along four dimensions:

1. **Confidence**: $c(p) = \prod_{i=0}^{L-1} \hat{T}(s_{i+1}|s_i, a_i)$, the cumulative transition probability reflecting empirical reliability.
2. **Quality**: $\bar{q}(p) = \frac{1}{L}\sum_i q(s_i, a_i)$, the average quality score along the path.
3. **Win Rate**: $\bar{w}(p) = \frac{1}{L}\sum_i \hat{w}(s_i, a_i)$, reflecting the expected terminal outcome.
4. **Reward**: $\bar{r}(p) = \frac{1}{L}\sum_i \hat{R}(s_i, a_i)$, the average per-step return.

The final **composite score** is computed via Min-Max normalized weighted sum:

$$\text{score}(p) = w_c \cdot \hat{c}(p) + w_q \cdot \hat{q}(p) + w_w \cdot \hat{w}(p) + w_r \cdot \hat{r}(p) \tag{8}$$

where $\hat{\cdot}$ denotes Min-Max normalization and $(w_c, w_q, w_w, w_r) = (0.3, 0.3, 0.3, 0.1)$.

Branch-to-Backup Recovery

During receding-horizon rollout, the actual state trajectory may diverge from the planned path due to stochastic transitions. Rather than discarding the entire plan and re-planning from scratch, we propose a **Branch-to-Backup (BTB)** recovery mechanism inspired by CPU branch target buffers.

Algorithm 1 Beam Search on ETG

Require: ETG G, start state s_0, beam width K, max depth D, score mode f, discount γ
Ensure: Recommended action a^*

$B_0 \leftarrow \{(s_0, \text{score} = 0, \text{path} = [], \text{prob} = 1.0)\}$
for $d = 1, \ldots, D$ **do**
  $\mathcal{C} \leftarrow \emptyset$
  for each $(s, c, p, \text{prob}_p) \in B_{d-1}$ **do**
    for each action a available at s in G **do**
      $\phi \leftarrow G.\text{get_stats}(s, a)$
      $\text{score}_n \leftarrow f(\phi) \cdot \gamma^d$
      for each $s' \in G.\text{next_states}(s, a)$ with $\hat{T}(s'|s, a) > \epsilon$ **do**
        $\text{prob}_n \leftarrow \text{prob}_p \cdot \hat{T}(s'|s, a)$
        $\mathcal{C} \leftarrow \mathcal{C} \cup \{(s', a, \text{score}_n, p \circ [(s, a)], \text{prob}_n)\}$
      end for
    end for
  end for
  $B_d \leftarrow$ Top-$K(\mathcal{C})$ by cumulative score
  if all paths in B_d reach terminal states **then**
    break
  end if
end for
$a^* \leftarrow$ first action of highest-scored path in B_D
return a^*, B_D

Switching Map Pre-computation. Before rollout, for each state s in the ETG, we pre-compute a set of **switching points**—alternative path branches that originate from s with quality scores exceeding a threshold θ_{score}:

$$\text{switching_map}(s) = \{(a, p_{\text{alt}}) : q(s, a) > \theta_{\text{score}},\ p_{\text{alt}} \text{ branches from } (s, a)\} \tag{9}$$

Deduplication by fork point ensures no redundant switching entries.

Two-Stage Divergence Matching. When the executed state s'_{actual} diverges from the planned state s'_{planned}:

1. **Exact Match**: Check if s'_{actual} exists as a switching point with identical state ID.
2. **Distance Fallback**: If no exact match, find the switching point whose state is within a distance threshold θ_d in the original continuous state space.

If a matching switching point is found, the planner seamlessly transitions to the alternative path, avoiding the computational cost of full re-planning.

Receding-Horizon Rollout

The complete planning pipeline operates as a receding-horizon controller inspired by Model Predictive Control (Camacho and Bordons 2007):

The rollout supports multiple **action selection strategies**: *best beam* (highest composite score), *epsilon-greedy* (exploration with probability ϵ), *highest transition probability*, and *subtree quality* (best average quality in the beam subtree).

Experiments

Experimental Setup

Environment. We evaluate on StarCraft II (PySC2) micromanagement scenarios (Vinyals et al. 2019) using the `MarineMicro` map family. Each scenario features controllable Marines fighting enemy Marines with varying army sizes and initial configurations.

Algorithm 2 Receding-Horizon Rollout with BTB

Require: ETG G, start state s_0, plan depth D_{plan}, beam width K, switching_map
Ensure: Trajectory with switching points

$s_{\text{cur}} \leftarrow s_0$
while s_{cur} is not terminal **do**
  // PLAN: Beam search from current state
  plan $\leftarrow$ BeamSearch(G, s_{cur}, K, D_{plan})
  $\text{path}_{\text{planned}} \leftarrow$ best path from plan
  // EXECUTE along planned path
  for each (s, a) in $\text{path}_{\text{planned}}$ **do**
    Execute a, observe s'_{actual}
    if $s'_{\text{actual}} \approx s'_{\text{planned}}$ (within θ_d) **then**
      $s_{\text{cur}} \leftarrow s'_{\text{actual}}$; continue
    else
      // DIVERGE: try BTB recovery
      sp $\leftarrow$ FindSwitchPoint(switching_map, s_{cur})
      if sp $\neq \emptyset$ **then**
        Switch to backup path; $s_{\text{cur}} \leftarrow s'_{\text{actual}}$
      else
        Break and re-plan from s'_{actual}
      end if
      break inner loop
    end if
  end for
end while

Table 2: **Map configurations used for data collection and evaluation.**

Map Configuration	Army Size	Variants
MarineMicro_MvsM_4	4v4	standard, mirror, dist
MarineMicro_MvsM_8	8v8	standard, mirror, dist

Data Collection. A hierarchical Q-Learning agent (SmartAgent) is trained against the built-in Very Hard AI bot across multiple configurations. Each episode records state sequences (via BK-Tree discretization), action sequences, step rewards, and game outcomes. We collect data across six map configurations (Table 2).

ETG Construction. We construct the ETG using the `Simple` configuration (state-only keys, no history context) with quality score weights 0.7 (future reward) and 30.0 (win rate). The resulting ETG contains 909 unique states and 55 unique actions, built from 13,834 evaluated state-action pairs.

Evaluation Metrics. We adopt five metrics for decision quality assessment:

- **Quality Score**: Average quality score of recommended actions.
- **Win Rate**: Historical win rate of recommended actions.
- **Coverage**: Percentage of states where the KG provides at least one valid recommendation.
- **Diversity**: Information entropy of the action distribution across recommendations.

Table 3: **Decision quality evaluation results on Marine-Micro_MvsM_4.**

Metric	Value
Total Evaluated State-Action Pairs	13,834
Unique States	909
Unique Actions	55
Quality Score	45.81
Win Rate	96.4%
Coverage	96.7%
Diversity (Entropy)	2.25
Confidence	16.1%

Table 4: **Simulation results over 100 episodes using ETG-guided planning.**

Metric	Value
Episodes	100
Wins	87
Losses	0
Max Steps Reached	13
Win Rate	**87%**
Avg. Episode Length	21.6 ± 13.8 steps
Avg. Quality Score	55.44
Avg. Win Rate (per-step)	98.35%

- **Confidence**: Average normalized visit frequency of recommended actions.

Decision Quality Evaluation

Table 3 presents the decision quality evaluation results. The ETG-based planner achieves a **96.4% historical win rate** on its recommended actions, indicating that the statistical aggregation effectively captures winning behavior patterns from the RL agent's experience. The **96.7% coverage** demonstrates that the ETG provides action recommendations for the vast majority of encountered states. The diversity score of 2.25 indicates meaningful action variety, avoiding degenerate single-action policies.

Trajectory Simulation

To evaluate the planning pipeline end-to-end, we simulate 100 complete episodes using the ETG with probability-based state transition sampling and roulette action selection.

Table 4 shows the results. The ETG-guided planner achieves an **87% win rate** over 100 simulated episodes with zero losses (13 episodes reached the maximum step limit). The average episode length of 21.6 ± 13.8 steps indicates efficient combat resolution. The per-step average win rate of 98.35% confirms that the planner consistently selects high-quality actions throughout each episode.

Analysis

ETG Structure Analysis. Figure 2 shows a portion of the ETG for the 4v4 scenario. The graph exhibits clear structural patterns: states with high win rates cluster in regions of the graph corresponding to advantageous spatial configurations, while terminal states (loss conditions) appear as dead-end nodes with red coloring. The edge density reflects the branching factor of available actions, and transition probability annotations highlight dominant action choices.

Figure 2: ETG Visualization

Nodes colored by win rate (green=high, red=terminal). Edge width proportional to visit count. Edge color gradient from red (low quality) to green (high quality).

Figure 2: **Interactive ETG visualization.** Each node represents an abstracted game state with size proportional to total visits. Edges are labeled with actions and transition probabilities.

Figure 3: Beam Search Tree

Hierarchical tree visualization of beam search paths. Each beam is color-coded. Nodes annotated with state ID and quality score. Edges labeled with actions and cumulative probability.

Figure 3: **Beam search planning tree for a single planning step.** Multiple candidate paths are explored in parallel, enabling comparison of alternative action sequences with different risk-reward profiles.

Beam Search Path Analysis.

Figure 3 illustrates a beam search tree generated for a planning query. Each beam represents an alternative action sequence, and the four-dimensional evaluation framework enables users to compare paths not only by expected reward but also by confidence (empirical reliability), quality, and win rate. This multi-faceted view supports nuanced decision-making that single-metric planning cannot provide.

Interpretability. A key advantage of the ETG approach is full decision traceability. For any recommended action a^* at state s, the system provides:

- The complete beam search tree showing all considered alternatives
- The four-dimensional evaluation scores for each candidate path
- The empirical statistics (visits, win rate, quality) supporting each choice
- The transition probabilities connecting each state-action pair

This transparency is fundamentally absent in neural network-based approaches and is critical for strategic analysis in game AI.

Discussion and Limitations

[TODO: Add baseline comparison experiments: random policy, greedy policy, VMG.] [TODO: Add ablation study: beam width $K \in \{1, 3, 5, 7\}$, with/without BTB, different scoring modes.] [TODO: Add cross-map generalization results (4v4 vs 8v8).]

Decoupled Architecture. The ETG framework achieves complete separation between data production (RL trajectory collection) and decision-making (graph-based planning). The planning process is **deterministic**: the same input state always produces the same recommended action, ensuring reproducibility. Model updates (rebuilding the ETG with new data) and planning algorithm improvements can proceed independently.

Limitations. The tabular nature of the ETG limits scalability to very large state spaces. The BK-Tree abstraction introduces information loss that may affect planning quality in scenarios with fine-grained state distinctions. The BTB recovery mechanism depends on the quality of pre-computed switching maps, which require sufficient historical data at divergence points. **[TODO: Expand limitations discussion with specific failure case analysis.]**

Conclusion

We presented the Experience Transition Graph (ETG), a lightweight, explicit world model for offline model-based planning in RTS micromanagement. By statistically aggregating RL trajectories into a directed graph with multi-dimensional edge attributes, the ETG provides a tabular approximation of the underlying MDP that supports interpretable, deterministic, and reproducible planning via beam search.

The four-dimensional path evaluation framework (confidence, quality, win rate, reward) enables nuanced action selection that balances empirical reliability with expected performance. The Branch-to-Backup recovery mechanism, inspired by CPU branch prediction, provides an efficient plan adaptation strategy that avoids full re-planning costs during receding-horizon rollout.

On StarCraft II micromanagement, the ETG planner achieves 96.4% historical win rate and 87% simulation win rate, demonstrating the effectiveness of purely experience-based planning without neural network training.

Future work includes: (1) scaling to larger scenarios via adaptive state abstraction; (2) integrating neural network value approximation for states with sparse visit data; (3) online ETG refinement with incremental updates from new trajectories; (4) extending to multi-agent cooperative scenarios with shared transition graphs.

References

Argenson, A.; and Dulac-Arnold, G. 2021. Model-Based Offline Planning. In *International Conference on Learning Representations (ICLR)*.

Burkhard, W. A.; and Keller, R. M. 1973. The Application of Metric-Space Techniques to the Study of Data Structures. *Journal of the ACM (JACM)*, 20(3): 437–452.

Camacho, E. F.; and Bordons, C. 2007. *Model Predictive Control*. Springer.

de Mendonça, R. F. C.; de Barros Costa, A. H. R.; and Prudêncio, R. B. C. 2018. Abstract State Transition Graphs for Model-Based Reinforcement Learning. In *IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, 3458–3465. IEEE.

Janner, M.; Qu, J.; Levine, S.; and Finn, C. 2021. Offline Reinforcement Learning as One Big Sequence Modeling Problem. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, 2063–2075.

Kuhn, H. W. 1955. The Hungarian Method for the Assignment Problem. *Naval Research Logistics Quarterly*, 2(1-2): 83–97.

Li, G.; Wei, W.; Chi, Y.; and Chen, Y. 2022. Settling the Sample Complexity of Model-Based Offline Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 38528–38541.

Ontañón, S.; Synnaeve, G.; Uriarte, A.; Richoux, F.; Churchill, D.; and Preuss, M. 2013. A Survey of Real-Time Strategy Game AI Research and Competition in StarCraft. *IEEE Transactions on Computational Intelligence and AI in Games*, 5: 293–311.

Samvelyan, M.; Virdee, T.; Sastry, A.; Liu, Z.; Zhang, S.; Jain, S.; Tarapore, D.; Forster, J.; and Farquhar, G. 2019. SMAC: The StarCraft Multi-Agent Challenge. In *Proceedings of the 18th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, 2186–2188.

Starre, R. N. A.; Saccenti, E.; and Ferreira, P. A. 2022. Model-Based Reinforcement Learning with State Abstraction: A Survey. *Machine Learning*, 111(11): 4075–4103.

Vinyals, O.; Babuschkin, I.; Czarnecki, W. M.; Mathieu, M.; Dudzik, A.; Chung, J.; Choi, D. H.; Powell, R.; Ewalds, T.; Georgiev, P.; et al. 2019. Grandmaster Level in StarCraft II Using Multi-Agent Reinforcement Learning. *Nature*, 575: 350–354.

Yin, M.; et al. 2024. Highway Graph to Accelerate Reinforcement Learning. *arXiv preprint arXiv:2405.11727*.

Zhan, X.; Hu, R.; Chen, Z.; Yang, B.; and Zhang, W. 2022. Offline Planning with Trajectory Pruning. In *International Joint Conference on Artificial Intelligence (IJCAI)*, 3420–3428.

Zhang, L.; Yang, G.; and Stadie, B. C. 2021. World Model as a Graph: Learning Latent Landmarks for Planning. In *International Conference on Machine Learning (ICML)*, 12591–12601.

Zhu, D.; Liu, Z.; Zhang, Z.; Zhang, Z.; Chen, Y.; and Yu, Y. 2023. Value Memory Graph: A Graph-Structured World Model for Offline Reinforcement Learning. In *International Conference on Learning Representations (ICLR)*.